

kaggle

REPORT INF 582 CHALLENGE



March 31st 2024

Matthieu Destrade, Rouault Charles-Edouard



1

INTRODUCTION

This paper presents our approach to addressing the "News Articles Title Generation" Kaggle competition. Our strategy involved thorough analysis of the dataset and the evaluation metric provided. We then focused on three key methods to achieve high scores in the competition. Firstly, we utilized pre-trained French summarization models. Secondly, we explored techniques to reduce the length of the texts to be summarized. Finally, we employed fine-tuning of the models to enhance performance. Through this paper, we elaborate on our strategies and discuss their effectiveness in improving the quality of generated titles.

2

ANALYSIS OF THE CHALLENGE

2.1 ANALYSIS OF THE DATA

We have a dataset of French press articles with their titles, separated in a training, a validation and a test sets, containing respectively 21401, 1500 and 1500 samples.

Set	Size	Average length - texts	Average length - titles	With quotes (%)
Train	21 401	350.3	32.1	0.296
Test	1500	353.7	-	-
Validation	1500	344.4	32.3	0.280

TABLE 1 – Dataset Review (length are in words)

The average of the texts lengths and titles lengths are consistent over the three datasets, and it is relevant to note that the maximum length of the texts is 3163 words, which we are going to have to take into account in our processing.

The texts are press articles, and the titles are constructed to be both a summary of the article and a catchy hook for the reader to be attracted to the article. We noticed that a very common method to create a catchy title was to quote someone's declaration : approximately 30% of the titles contain a quote.

2.2 ANALYSIS OF THE TASK

In this challenge, we are tasked with generating titles out of a text, which is a "sequence to sequence" process, like translation. More specifically, we are expected to reproduce, as faithfully as possible, the original title that the author of the article gave. The difference between the generated title and the real one is evaluated using the ROUGE-L F1-score metric.

The amount of text in the train dataset cannot be compared with the amount of data in typical NLP language models (226 000 for the xsum dataset, for instance). Training a new model from scratch would therefore never reach state-of-the-art performance because of the fewer data points. However, the dataset is large enough to fine-tune a pre-trained model. A specificity of the challenge is also the monolinguality of the dataset, which is fully in French, which is something to take into account in our research to achieve the best performance.

All of these considerations incited us, after initial tests, to explore already existing French summarization models.

3 APPROACHES

3.1 IMPLEMENTATION OF OUR BASELINES IDEAS

Before testing the state-of-the-art (SOTA) models for title generation, we decided to investigate the problem of title generation with simple scripts, in addition to the EXT-Oracle and the LEAD baseline scripts provided, to gain a better understanding of how to perform well.

As the ROUGE-L metric depends on the longest common sub-sequence, we wanted to investigate whether we would get a higher baseline score if we increased the length of the title generated. Indeed, with more words, the probability of achieving a longer common sub-sequence could be higher. We tested with all the text as a title (called the “raw text” baseline) and got a worse score than the other baselines. This is because the ROUGE-L F1-score metric takes precision into account as well.

We then attempted to find ways to distinguish between a catchy title that could be human-generated and a purely summarizing title. An idea was that a catchy quote is often used in the title. Indeed, around 30% of the titles of the validation and training data contain brackets. So, as another baseline attempt, we systematically extracted all of the sentences with the quotes. We concatenated them with the first sentence of the text and got a better baseline score than the one obtained with LEAD-1.

Method	ROUGE-L F1-score
LEAD-1	0.156
EXT-ORACLE	0.316
Raw text	0.100
Quote	0.163

TABLE 2 – ROUGE-L F1-scores of Baselines Methods

The score of the EXT-ORACLE method is very convincing. However, this method cannot be practically applied as it necessitates prior knowledge of the titles of the texts. The score of the "quote" method is still far from what we can hope to achieve. The "raw text" method is not very efficient either.

3.2 USE OF PRETRAINED MODELS

After investigating our baseline ideas, we analyzed the existing literature about this task. We found several multi-language models that were trained for title generation, but the SOTA performances for the task of generating titles from French texts were achieved by models trained on French data : FlauBERT, CamemBERT, BARThez.

Models	Rouge-L score
BARThez	36.03
CamemBERT2CamemBERT	30.83
mBARThez	36.41

TABLE 3 – Comparison of the ROUGE-L scores for title generation for literature models on a dataset different than ours, copied on [1]

We tried several of these models fine-tuned for summarization tasks on our validation set, namely :

— csebuetnlp/mT5_multilingual_XLSum

- moussakam/barthez-orangesum-abstract
- mrm8488/camembert2camembert_shared-finetuned-french-summarization
- bofenghuang/flan-t5-large-dialogsum-fr

Their resulting rouge-L scores over the 50 first examples of the validation set are presented in the table 4 :

Models	Rouge-L F1-score
BARThez-orangesum-abstract	0.2306
CamemBERT2CamemBERT	0.2072
Flan	0.2001
mT5 multilingual XLsum	0.2323

TABLE 4 – Comparison of the ROUGE-L F1-scores for title generation for literature models on our test dataset

The best performing model is `mT5_multilingual_XLSum`, followed by `barthez-orange-sum-abstract`. In practice, on the test set, the best results were obtained with Barthez.

3.3 LENGTH REDUCTION

With these models, we were faced with a problem : they have a maximum length of input they can process. For instance, Barthez can only process inputs whose size (i.e., the number of tokens) is less than 1024, and Camembert can only process inputs whose size is less than 512. However, in our test and validation sets, some instances are longer than those values.

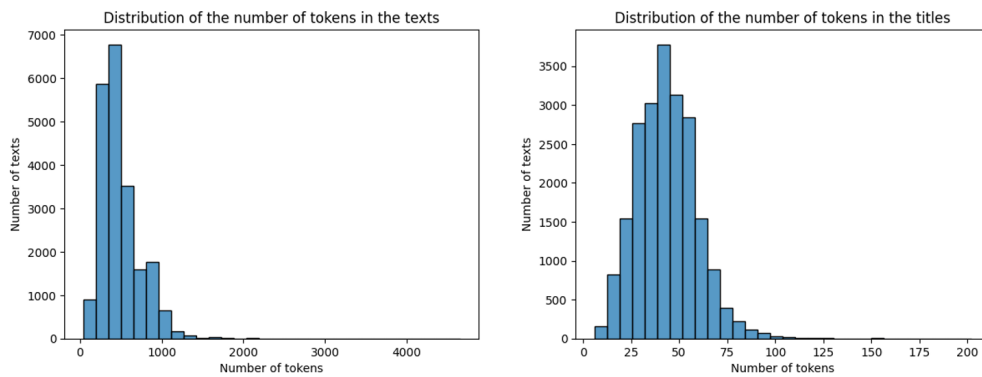


FIGURE 1 – Distribution of the length of the texts and the titles in the training set

The easiest solution is then to truncate the texts to the maximum length. However, doing so discards a lot of available information, and we believed that selecting the sentences we feed into the models more accurately could lead to better results.

In order to try to improve our results, we therefore started by trying the following procedure : if a text is too long, we split it into several parts of computable size, we generate a summary for each of these parts, and we finally concatenate the summaries. The results of this method were mitigated on single examples and sometimes greatly improved results, but they did not lead to an improvement in performance over the whole dataset.

We therefore applied another idea : we preprocessed the texts using an extractive summarization technique. To this end, we compute the cosine similarities between sentences in the text and select the ones with the biggest similarity with the others. We keep the maximum number of sentences possible, given the maximum

input text size that is computable. This method was very promising, and we observed a massive improvement of the rouge-L score when using it on some individual texts. However, on the whole dataset, it led to a small reduction of the score. We believe that designing a way to determine if it should be used or not on a text could therefore drastically improve results.

Finally, another idea we tried to improve our results was to apply the EXT-ORACLE method using our generated summaries. The idea was that if our models produced meaningful summaries, EXT-ORACLE might be better at reusing the same words as the ones in the text, and that this could lead to better rouge-L scores. However, it was not the case.

The performances of these methods, using the best model we managed to get, on the 150 first samples of the validation set, are presented in the table 5 :

Approach	Rouge-L F1-score
Model alone	0.2488354478820943
Splitting and concatenating	0.21588701195212234
Extractive summary	0.2473937638335417
Adding EXT-ORACLE	0.20356866779680835

TABLE 5 – Comparison of the ROUGE-L F1-scores for title generation for the different approaches described on our test dataset

Therefore, none of these methods allowed us improve our results on the whole test set.

3.4 FINE-TUNING

Another way we explored to improve our results, other than by preprocessing or postprocessing our data, was to fine-tune the models we use. In order to fine-tune the models we selected, we used two different tools.

- TRANSFORMERS MODULE

First, we used a built-in function from the `transformers` module : `run_summarization.py` (available in `transformers/examples/pytorch/summarization/run_summarization.py`). It allows us to easily fine-tune any model by specifying as arguments the name of the model, the train and validation datasets, and the batch size (among some other arguments related to save and test frequencies). For certain models, however, we had some trouble running this function and had to modify parameters directly in their configuration files. It was the case for the `barthez-orange-sum-abstract`, for which we had to update the `config.json` file, copying one of the traditional Barthez models.

We then tried to fine-tune several models.

We started with the model that had the best results initially, namely `barthez-orange-sum-abstract`. However, we quickly observed that the loss and Rouge-L value over the validation set were only respectively increasing and decreasing through training.

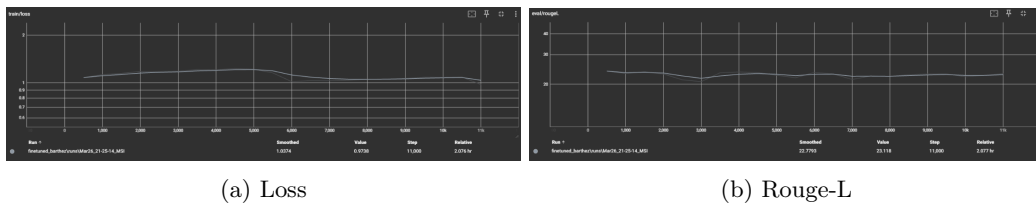


FIGURE 2 – Loss and Rouge-L during training of `barthez-orangesum-abstract` on our train set

We observed no improvement in the rouge-L score on the validation set.¹

After some tests with the training data, we realized that the loss over the train set was abnormally low and suspected that the model had already been trained with our training data. Indeed, after verification, the orange sum dataset is actually the one we are provided with for the Kaggle challenge. Since this model could not be improved with our train set, we tried to improve it a bit by fine-tuning it over our validation set. Monitoring the performances over the train set to avoid overfitting, we were able to improve it after two epochs of training (rouge-L score on the test set was improved from 2.3242 to 2.3324). However, we believed that even better performances could be obtained by training other models, initially trained on other datasets, on our training set.

We therefore tried to fine-tune the second-best-performing model, namely `mT5_multilingual_XLSum`. However, this model is far bigger than Barthez (582M parameters instead of 140M), and using our computers, we were not able to compute fine tuning in a reasonable time. The same issue arose with other models we tried to fine-tune, such as `mBarthez` or `flan-t5-large-dialogsum-fr`. A solution to this problem could be to freeze the first layers of these models and only train the more superficial ones, which comes with the risk of limiting the fine-tuning efficiency. We did not implement it.

We were still able to fine-tune another model that is roughly the same size as Barthez : `camembert2camembert` (140M parameters).

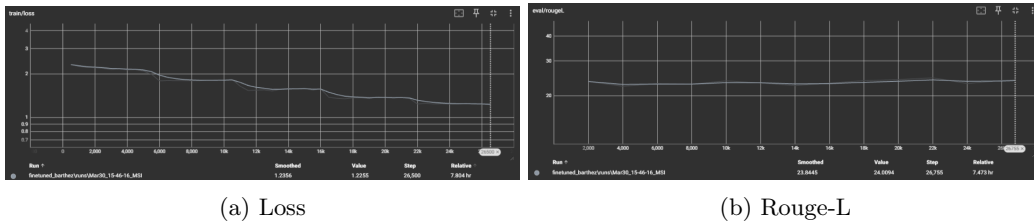


FIGURE 3 – Loss and Rouge-L during training of `camembert2camembert` on our train set

We successfully fine-tuned the model and managed to get better rouge-L scores on the validation (0.2488354478820943 on the 50 first texts) and test sets (0.23455).

• FAIRSEQ

Secondly, we tried to fine-tune using fairseq, a sequence modeling toolkit [2]. We decided to follow the finetuning approach proposed on the github of the BARThez module.

As a first step, we constructed our own dictionary using byte pari encoding. This method creates a dictionary based on the frequency of the groups of letters, so the data it has been trained eases the efficacy of the computation. Other dictionaries are often biased with a lot more English than other languages, so they are not optimal for a dataset with French text only.

We decided to go for a dictionary of 32 000 tokens, in order to optimize the handling of unseen or rare words, and to speed up the computational power by compressing the embeddings.

The BARThez tokenizer is already trained on a monolingual French dataset, but in order to avoid as much as possible the handling of unknown tokens, we decided to generate our own Sentence Piece dictionary with the training, validation, and testing data.

Then, we trained the BARThez model using the fairseq module, but our computational power didn't allow us to go further than 10 epochs training, which didn't improve our performance : it went down to 0.18 on the

1. The train graphs are available as tfevents files in the .zip file provided with this report in `finetuned_barthez/runs`

test set. Out of computational choices, we decided not to go further with this approach.

4

CONCLUSION

To conclude, we developed several strategies to compete in the "News Articles Title Generation" Kaggle competition. All of them are based on HuggingFace French summarization models, and especially barthez-orangesum-abstract and camembert2camembert_shared-finetuned-french-summarization.

The first methods we tried mostly involved preprocessing and postprocessing the initial data, or the results produced by these models, and had mitigated performances. In order to really improve our scores, we had to fine-tune the models. Since barthez-orange-sum-abstract was already trained on our dataset, there was little room for improvement with this model, but using camembert2camembert_shared-finetuned-french-summarization we were able to get good performances.

We believe that they could even be further improved by fine-tuning this model more accurately, since at some point during training we reached rouge-L scores of almost 0.25 on the validation dataset, but did not save the corresponding model. Additionally, the use of a preprocessing step to compute extractive summaries of the texts and reduce their lengths seems promising and could yield interesting results with more complex conditions of application on the texts.

It is important to observe that the computational constraints posed limitations on the methods we opted to investigate, underscoring the imperative to minimize computational expenses wherever feasible.

5

REFERENCES

- 1 Eddine, Moussa Kamal, Antoine J.-P. Tixier, et Michalis Vazirgiannis. *BARThez : a Skilled Pretrained French Sequence-to-Sequence Model*. arXiv, 9 février 2021. <https://doi.org/10.48550/arXiv.2010.12321>
- 2 Ott, Myle, Sergey Edunov, Alexei Baevski, Angela Fan, Sam Gross, Nathan Ng, David Grangier, et Michael Auli. *fairseq : A Fast, Extensible Toolkit for Sequence Modeling*. arXiv, 1 avril 2019. DOI <http://arxiv.org/abs/1904.01038>